Development of an Arkanoid Game in C++ Using SFML and Object-Oriented Principles

Muhammad Haseeb
Department of Computer Systems Engineering
University of Engineering and Technology, Peshawar

August 21, 2024

Abstract

This report presents the design and implementation of an Arkanoid-style game built with C++ and SFML. The project demonstrates object-oriented programming (OOP) principles such as inheritance, abstraction, and polymorphism. Key features include real-time paddle control, ball physics, destructible bricks of varying durability, collision detection using axis-aligned bounding boxes (AABBs), and immersive audio-visual feedback. The modular design ensures scalability and future extensions.

1 Introduction

The Arkanoid genre is a classic paddle-and-ball game, making it ideal for learning game development concepts. In this project, the game is implemented using the Simple and Fast Multimedia Library (SFML), which provides tools for rendering, audio, and input handling. The main objective was to apply OOP concepts for a maintainable and extensible codebase.

2 Objectives

- Create a functional Arkanoid game with classic mechanics.
- Apply inheritance and polymorphism to game entities.
- Implement robust collision detection.
- Develop levels with progressive difficulty.
- Integrate scoring, HUD, and audio-visual feedback.

3 Game Mechanics

3.1 Paddle Control

The paddle moves horizontally at the bottom of the screen, controlled by arrow keys.

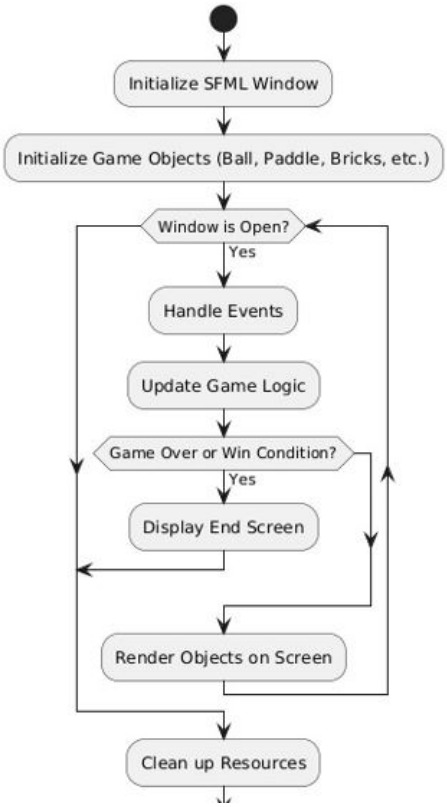

Figure 1: Paddle movement via keyboard input.

3.2 Ball Dynamics

The ball follows a velocity vector and reflects upon collisions. Missing the paddle costs a life.

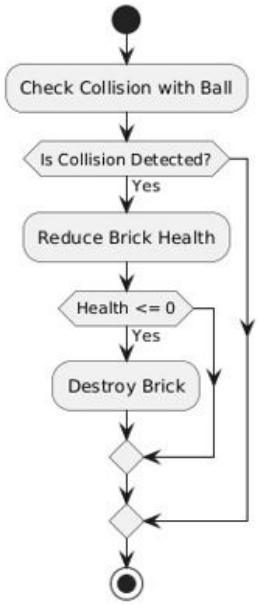

Figure 2: Ball bouncing off the paddle.

3.3 Brick Hierarchy

Bricks have three durability levels:

- **Bronze:** destroyed in 1 hit
- **Silver:** destroyed in 2 hits
- **Gold:** destroyed in 3 hits

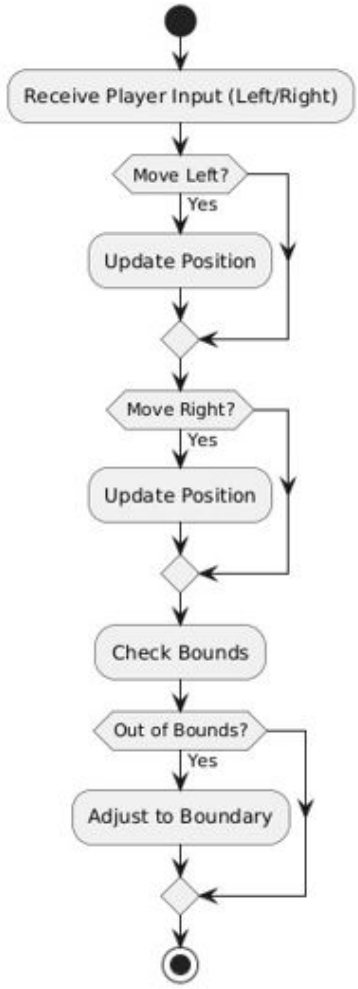

Figure 3: Bronze, Silver, and Gold bricks.

3.4 Scoring and Levels

Players score points by destroying bricks. Levels increase in difficulty with new layouts. Game over occurs when lives reach zero.

3.5 Audio-Visual Feedback

- Custom background textures
- Sound effects for collisions and events
- HUD showing score, lives, and level

4 Design and Implementation

4.1 Class Hierarchy

- `Brick`: Base class for all brick types
- `BronzeBrick`, `SilverBrick`, `GoldBrick`
- `Paddle`: Handles player input and movement
- `Ball`: Manages velocity and collision

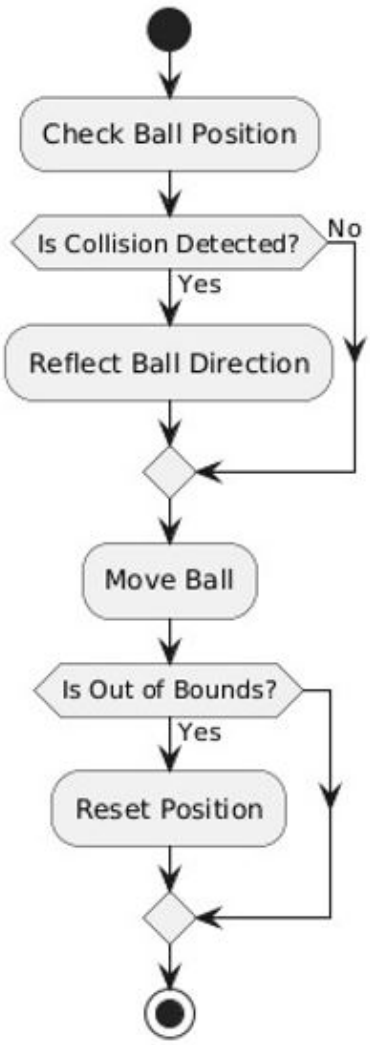

Figure 4: Simplified class diagram.

4.2 Core Methods

- `initializeBricks()` – generates levels
- `move()` – updates entity positions
- `handleCollision()` – processes collisions

4.3 Collision Detection

Implemented using AABBs for efficiency.

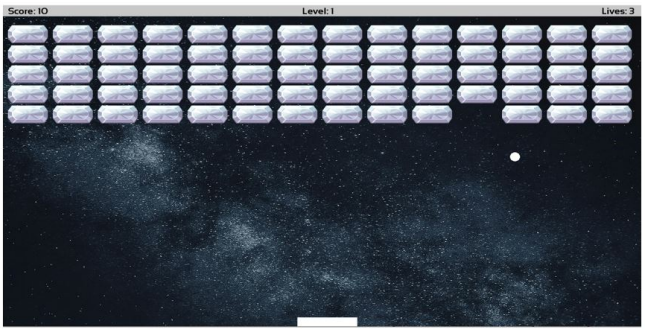

Figure 1:Archnoid Game

Figure 5: Ball–brick collision detection.

5 OOP Concepts in Practice

- **Classes & Objects:** Encapsulation of entities.
- **Inheritance:** Brick subclasses reuse base logic.
- **Polymorphism:** Virtual methods like `draw()` and `hit()`.
- **Abstraction:** Clean interfaces hide SFML internals.

6 Challenges and Solutions

- Collision precision handled with AABBs.
- Brick durability managed with counters.
- Level design simplified via array-based layouts.
- Audio synchronized with event handlers.

7 Conclusion

This Arkanoid project demonstrates OOP-based game development using C++ and SFML. The modular design allows for future features like power-ups, more levels, and new game modes.